



Jakarta EE

“

Brinda a los desarrolladores un conjunto integral de especificaciones **abiertas y neutrales respecto al proveedor**, utilizadas para desarrollar aplicaciones Java modernas y nativas de la nube desde cero.

De Java EE a Jakarta EE

En 2019, Oracle transfirió Java EE a la **Eclipse Foundation**. Por la marca registrada "Java", desde Jakarta EE 9 (2020) todas las especificaciones tuvieron que renombrar su espacio de nombres de `javax.*` a `jakarta.*`.

Una migración ya resuelta

Herramientas como **OpenRewrite** automatizan hoy buena parte de ese cambio de imports. Lo que en 2020 era una gran incertidumbre, hoy es un problema técnico ya solucionado.

Por qué importa hoy

Spring Boot se apoya en anotaciones `jakarta.*`, y buena parte de la administración pública española sigue sobre servidores Jakarta/Java EE. **Más en la página 2.**

CONTENIDO

02 Jakarta EE hoy y mañana

04 APIs de Jakarta EE

06 Versiones y perfiles

08 Servidores de aplicaciones

03 Jakarta EE vs Spring Boot

05 Especificaciones más comunes

07 Arquitectura MVC

OBJETIVO DE HOY

Entender qué problema resolvió Jakarta EE, cómo ha evolucionado hasta la versión 11 y por qué sigue siendo relevante en un ecosistema dominado por Spring.

Jakarta EE hoy y mañana

arquitecturajava.com — "¿Qué es Jakarta EE?"

De Java EE a Jakarta EE: qué cambió de verdad

El cambio más disruptivo fue el paso del espacio de nombres `javax.*` a `jakarta.*`: Oracle conservó la marca "Java", así que desde Jakarta EE 9 todas las especificaciones tuvieron que renombrar sus paquetes. Migrar un proyecto de Java EE 8 a Jakarta EE moderno implica, en la práctica:

- 1 Actualizar los imports (`javax.persistence`, `javax.servlet`, `javax.ejb` ...) a su equivalente `jakarta.*`
- 2 Subir el servidor de aplicaciones a una versión compatible con Jakarta EE 9+
- 3 Auditar las librerías de terceros que aún dependan del namespace antiguo



Herramientas como **OpenRewrite** automatizan buena parte de esta migración: "hoy es un problema resuelto, no un obstáculo".

Dónde estamos y hacia dónde vamos

ACTUAL · 2025

Jakarta EE 11

- **Jakarta Data 1.0** — patrón repositorio al estilo Spring Data, ahora como estándar
- **Virtual Threads** — soporte vía Jakarta Concurrency sobre Java 21
- Base mínima: **Java SE 17 y 21 LTS**
- Actualizaciones fuertes en CDI 4.1, Persistence 3.2, Security 4.0 y JAX-RS 4.0

EN DESARROLLO · 2026

Jakarta EE 12

- Soporte a nivel de API para **Java SE 21**, ejecutable sobre Java SE 25
- Nuevas especificaciones: **Jakarta Query** y mejoras en Jakarta NoSQL
- Posible convergencia con **MicroProfile** (REST Client + JAX-RS unificados)
- Ciclo de publicación aproximadamente bienal

¿Por qué sigue importando en la era de Spring y la IA?

La base de Spring Boot

Anotaciones tan cotidianas como `jakarta.persistence.Entity`, `jakarta.validation.constraints` o `jakarta.servlet` son, literalmente, especificaciones de Jakarta EE.

Legado y sector público

Buena parte de la administración pública española y grandes corporaciones mantienen sistemas Java EE clásicos sobre WildFly, WebLogic o WebSphere: modernizarlos es una salida profesional real.

Portabilidad entre proveedores

Al ser un estándar con múltiples implementaciones, sigue siendo la opción de referencia cuando la portabilidad entre servidores es un requisito de arquitectura.

Jakarta EE frente a Spring Boot

No son rivales: Spring Boot usa piezas de Jakarta EE por debajo (`@Entity` , `@Valid` , `jakarta.servlet` ...). Pero a la hora de elegir arquitectura — o de defenderte en una entrevista — conviene tener clara la diferencia de enfoque.

ASPECTO	JAKARTA EE	SPRING BOOT
Naturaleza	Estándar abierto, con varias implementaciones (Eclipse Foundation)	Framework de un proveedor (Broadcom / VMware)
Inyección de dependencias	CDI — <code>@Inject</code>	Spring DI — <code>@Autowired</code> , inspirado en CDI
Persistencia	Jakarta Persistence (JPA)	Spring Data JPA, construido sobre JPA
Capa web / REST	Servlet, Faces (JSF), JAX-RS	Spring MVC, Spring WebFlux
Configuración	Especificación + descriptor del servidor	"Convención sobre configuración" + autoconfiguración
Servidor de despliegue	Externo — WildFly, TomEE, GlassFish... (o standalone)	Embebido — Tomcat/Netty empaquetado en el propio <code>.jar</code>
Portabilidad	Alta: el mismo código corre en cualquier servidor certificado	Ligada al ecosistema y a las librerías de Spring
Dónde se ve más	Legado empresarial, sector público, banca	Startups, microservicios, proyectos nuevos



En una entrevista, saber que **Spring reutiliza el estándar de Jakarta EE** — no lo sustituye — es la respuesta que distingue a quien entiende el ecosistema de quien solo sabe usar un framework.

Introducción a las APIs de Jakarta EE

jakarta.ee/specifications

Jakarta EE agrupa más de 35 especificaciones. Aquí están organizadas por el papel que cumplen dentro de una aplicación empresarial, para que sea más fácil ubicar cuál necesitas en cada momento.

● Web y presentación

Faces (JSF)

Server Pages (JSP)

Servlet

Standard Tag Library (JSTL)

MVC

RESTful Web Services (JAX-RS)

Expression Language (EL)

WebSocket

● Inyección, contexto y componentes

Context & Dependency Injection (CDI)

Enterprise Java Beans (EJB)

Interceptors

Managed Beans

Annotations

Deployment

● Persistencia y datos

Persistence (JPA)

NoSQL

Bean Validation

● Seguridad

Security

Authentication

Authorization

● Mensajería y concurrencia

Messaging (JMS)

Concurrency

Batch

Mail

● Integración y conectividad

Connectors (JCA)

Transactions (JTA)

Management

Config

● Servicios web, XML y JSON (legado / interoperabilidad)

Enterprise Web Services (JAX-WS)

SOAP with Attachments

Web Services Metadata

XML Binding

XML Registries

XML RPC

XML Web Services Specification

RPC

JSON Binding (JSON-B)

JSON Processing (JSON-P)

Las especificaciones más comunes

Breve descripción de las siete especificaciones que aparecen con más frecuencia en aplicaciones Jakarta EE (antes conocido como Java EE).



JSP

Jakarta Server Pages

Tecnología que permite crear contenido dinámico en páginas web combinando plantillas HTML con código Java insertado. Se usa para construir vistas, de forma similar a los servlets, pero con una separación más clara entre lógica de negocio y presentación.



JSTL

Jakarta Standard Tag Library

Biblioteca de etiquetas que simplifica el uso de Java dentro de las páginas JSP: bucles, condicionales, manejo de XML, internacionalización y datos, evitando escribir demasiado código Java en la capa de presentación.



JSF

Jakarta Faces

Framework para construir interfaces de usuario basadas en componentes reutilizables (formularios, botones, tablas...), enlazados directamente con los modelos de datos del backend. Ideal para aplicaciones de tipo formulario.



JPA

Jakarta Persistence API

Especificación de mapeo objeto-relacional (ORM) que permite trabajar con bases de datos relacionales usando objetos Java en vez de SQL directo, abstrayendo el CRUD mediante anotaciones y entidades.



EJB

Jakarta Enterprise Beans

Componentes del lado del servidor que encapsulan la lógica de negocio, con servicios de transacciones, concurrencia y seguridad ya integrados. Pensados para aplicaciones empresariales distribuidas y de gran escala.



CDI

Contexts & Dependency Injection

Mecanismo de inyección de dependencias y control de contexto: el contenedor gestiona automáticamente el ciclo de vida de los objetos y permite inyectar recursos o servicios donde se necesiten.



Servlets

Jakarta Servlet

Clases Java que se ejecutan en un servidor web, reciben peticiones HTTP y generan respuestas dinámicas. Son el núcleo de las aplicaciones web en Jakarta EE y la base sobre la que se apoyan JSP y JSF.



Servlet, JSP/JSTL, JSF, JPA, EJB y CDI son, precisamente, las piezas mínimas para montar un MVC clásico en Jakarta EE — ver el desglose por capas en la [página 7](#).

Versiones

1999		J2EE 1.2
2001		J2EE 1.3
2003		J2EE 1.4
2006		Java EE 5
2013		Java EE 7
2017		Java EE 8
2019		Jakarta EE 8 Última versión bajo el nombre "Java EE" a nivel de paquetes
2020		Jakarta EE 9 Migración del espacio de nombres javax.* → jakarta.*
2021		Jakarta EE 9.1
2022		Jakarta EE 10
2025		Jakarta EE 11 — actual Jakarta Data 1.0, Virtual Threads, base Java SE 17/21
2026		Jakarta EE 12 En desarrollo — ver página 2

Perfiles de Jakarta EE

No todas las especificaciones son igual de populares, y no todas las comunidades o empresas Java las adoptan por completo. Spring Boot y Micronaut, por ejemplo, ofrecen capas de abstracción sobre Jakarta Persistence pero definen APIs REST a su manera. Cuando una empresa sí adopta Jakarta EE por completo, puede certificar sus productos con alguno de estos tres perfiles:

PERFIL

Core

Conjunto minimalista de especificaciones, pensado para la creación de servicios REST.

PERFIL

Web

Conjunto de especificaciones orientado a construir aplicaciones web completas.

PERFIL

Platform

Conjunto completo de características, para productos que cubren cualquier tipo de aplicación Java Enterprise.



Los perfiles son acumulativos en complejidad: un servidor certificado en **Platform** puede ejecutar cualquier aplicación pensada para **Web** o **Core**, pero no siempre ocurre al revés.

Un MVC mínimo con Jakarta EE

Para construir una aplicación web con el patrón Modelo-Vista-Controlador usando Jakarta EE, estas son las especificaciones mínimas recomendadas para cada capa, junto con sus alternativas.

CAPA	ESPECIFICACIÓN	ROL / ALTERNATIVA
Controlador	Base Servlet	Recibe las peticiones HTTP, interactúa con el modelo y redirige a la vista adecuada (normalmente una JSP).
	Opcional RESTful Web Services (JAX-RS)	Alternativa cuando la capa de controlador expone una API REST en lugar de vistas server-side.
Vista	Base Server Pages (JSP) + JSTL	Genera HTML dinámico con plantillas; JSTL evita que la lógica Java invada la presentación (bucles, condicionales, formularios).
	Opcional Faces (JSF)	Enfoque basado en componentes de UI reutilizables enlazados al modelo; más completo, pero más pesado para una app sencilla.
Modelo	Base Persistence (JPA)	Define entidades que representan las tablas y gestiona el CRUD por ORM, sin SQL explícito.
	Base CDI	Gestiona la inyección de dependencias de los beans "controlador" o "servicio" que necesite la aplicación.
	Recomendado Transactions (JTA)	Mecanismo estándar para transacciones distribuidas entre la lógica de negocio y la base de datos.
	Recomendado Bean Validation	Valida los datos de entidades JPA o de formularios enviados desde la vista.
	Según escala EJB	Para lógica de negocio robusta: transacciones distribuidas, concurrencia o aplicaciones de gran escala.



Flujo de una petición

De forma simplificada, así recorre una petición HTTP las tres capas hasta convertirse en una respuesta.



Servidores de aplicaciones

jakarta.ee/compatibility

Listado de los servidores de aplicaciones compatibles más utilizados, donde puede desplegarse una aplicación Jakarta EE.



SERVLET · JSP

Contenedor de servlets ligero y el más extendido del ecosistema Java. Punto de partida habitual para aprender Servlet, JSP y JSTL.

No implementa todas las especificaciones de Jakarta EE

tomcat.apache.org

TomEE

WEB · PLATFORM

Apache Tomcat ampliado por la Apache Software Foundation con las piezas que faltan para cubrir Jakarta EE: CDI, JPA, EJB, JAX-RS y más, en distintos perfiles de empaquetado.

tomee.apache.org

PLATFORM

Servidor de aplicaciones de Red Hat (antes JBoss AS), de código abierto, con arranque rápido y soporte muy completo del Jakarta EE Platform Profile.

wildfly.org

GlassFish

PLATFORM · REFERENCIA

Implementación de referencia oficial de Jakarta EE, mantenida por la Eclipse Foundation. Es donde se validan primero las nuevas especificaciones de cada versión.

glassfish.org

Jakarta EE sigue evolucionando de forma abierta bajo la **Eclipse Foundation**, con el código como base del proceso de especificación — cualquiera puede seguir su desarrollo en github.com/jakartaee.